



Carrera: Analista Programador
Asignatura: Introducción a la Programación Segura

INFORME DE ACTIVIDAD PRÁCTICA

Sistema seguro de registro de pacientes en Python

Validación de datos, privacidad, integridad de archivos,
pruebas automatizadas y análisis de seguridad

Integrantes

Marcelo Navarro
Roberto Cortés
Diego Ávila

Docente

Miguel Angel Chacana González

24 de julio de 2026

*Proyecto académico desarrollado con datos completamente ficticios.
No debe utilizarse para almacenar información médica real.*

1. Objetivo y alcance

El objetivo de la actividad fue construir una aplicación de consola en Python que permitiera registrar pacientes ficticios de manera controlada. Antes de guardar un dato, el programa lo valida y sanitiza. También genera un identificador seudónimo, muestra reportes sin diagnóstico y detecta modificaciones del archivo JSON mediante SHA-256.

El proyecto tiene un alcance solamente académico. Se trabajó con módulos básicos de Python, funciones pequeñas, archivos JSON y un menú de texto. Para revisar el resultado se utilizaron pytest, pytest-cov, Bandit, GitHub Actions, SonarQube Cloud y GitHub Copilot.

Resultado final: 47 pruebas aprobadas, 93 % de cobertura local, 0 hallazgos de Bandit y Quality Gate aprobado en SonarQube Cloud.

2. Organización del proyecto

Archivo o carpeta	Responsabilidad
app.py	Menú, registro, búsqueda, reporte y control de errores.
validaciones.py	Validación del RUT, edad y correo; sanitización de nombre y diagnóstico.
privacidad.py	Identificador seguro y creación de vistas anonimizadas.
almacenamiento.py	Lectura y escritura JSON, escritura atómica y verificación SHA-256.
tests/	Pruebas de validación, privacidad, almacenamiento y funcionamiento del menú.
.github/workflows/	Ejecución automática de pytest, cobertura, Bandit y SonarQube Cloud.

Resumen de cumplimiento

Actividad	Resultado
Aplicación con datos ficticios	Cumplido
Pruebas automatizadas	47 aprobadas
Cobertura local	93 %
Bandit	0 hallazgos
SonarQube Cloud	Quality Gate aprobado
GitHub Copilot	Propuesta revisada y documentada

3. Matriz de riesgos

Antes de programar se identificaron riesgos sencillos relacionados con la entrada de datos, la privacidad y los archivos. La tabla siguiente muestra el control aplicado y el principio de confidencialidad, integridad o disponibilidad que se busca proteger.

Riesgo	Impacto	Prob.	Control aplicado	CIA
Caracteres no permitidos	Datos incorrectos o contenido malicioso.	Media	Sanitización y límites de longitud.	I
Edad inválida	Registro inconsistente.	Media	Conversión controlada y rango de 0 a 120.	I
Exposición de datos	Pérdida de privacidad.	Alta	Reporte anonimizado y diagnóstico excluido.	C
RUT usado como ID	Facilita relacionar el registro con una persona.	Media	ID aleatorio generado con secrets.	C
Modificación del JSON	Pérdida de confianza en los datos.	Media	SHA-256 y comparación segura del hash.	I
JSON corrupto o ausente	El sistema no puede iniciar.	Media	Creación automática y excepciones controladas.	D
Dependencia falsa	Ejecución de código no confiable.	Baja	Nombres y versiones revisadas y fijadas.	C/I/D
Sugerencia de IA incorrecta	Prueba o código inseguro.	Media	Revisión humana, pruebas y análisis estático.	C/I/D

C = confidencialidad; I = integridad; D = disponibilidad. La prioridad principal fue evitar la exposición de información y detectar cambios no autorizados del archivo.

4. Aplicación simplificada de OWASP SAMM

Función	Aplicación en el proyecto
Gobierno	Reglas de trabajo: datos ficticios, sin secretos y revisión de las sugerencias de IA.
Diseño	Matriz de riesgos, datos sensibles identificados y módulos separados.
Implementación	Validaciones, sanitización, errores controlados, ID seguro y escritura atómica.
Verificación	Pruebas manuales, pytest, cobertura, Bandit y SonarQube Cloud.
Operaciones	Creación automática de archivos y detección de alteraciones.

Entendimos SAMM como una forma de ordenar y mejorar las prácticas de seguridad. El proyecto se encuentra en un nivel inicial, pero ya incorpora controles, evidencias y automatización básica.

5. Aplicación simplificada de Microsoft SDL

Etapas	Trabajo realizado
Capacitación	Revisión de CIA, privacidad, dependencias e IA responsable.
Requisitos	Registro, búsqueda, reporte, integridad y rechazo de entradas inválidas.
Diseño	Arquitectura modular y riesgos definidos antes de programar.
Implementación	Funciones pequeñas, secrets, SHA-256 y manejo de excepciones.
Verificación	47 pruebas, 93 % de cobertura local, Bandit y SonarQube Cloud.
Liberación	Sin datos generados, secretos ni entorno virtual en la entrega.
Respuesta	Alerta crítica y cierre cuando el JSON no coincide con su hash.

6. Aplicación de la tríada CIA

Principio	Aplicación
Confidencialidad	Se ocultan nombre, RUT y correo; el diagnóstico no aparece en los reportes.
Integridad	Los datos se validan, se escriben de forma atómica y se verifican con SHA-256.
Disponibilidad	Se crean carpetas, se controlan errores y el menú continúa después de datos inválidos.

7. Evidencias del funcionamiento

Se utilizaron solamente datos ficticios. Las capturas muestran el registro, el identificador generado, el reporte anonimizado y los archivos creados por el programa.

```

●●● Evidencia - registro

REGISTRO DE TRES PACIENTES FICTICIOS
=====
Paciente 1: Paciente registrado correctamente.
Identificador asignado: PAC-0JHTECCGA9KF
Paciente 2: Paciente registrado correctamente.
Identificador asignado: PAC-R4AGJH6WMCPM
Paciente 3: Paciente registrado correctamente.
Identificador asignado: PAC-P2D7PII3CXRI

Los identificadores cambian en cada ejecución por usar secrets.

```

Figura 1. Registro de pacientes ficticios e identificadores generados.

```

●●● Evidencia - reporte anonimizado

REPORTE ANONIMIZADO
=====

Paciente número 1
-----
ID: PAC-0JHTECCGA9KF
Nombre: A** P*****
RUT: *****-5
Edad: 31
Correo: a*****@example.com
Fecha de registro: 2026-07-21T02:46:28.363996+00:00

Paciente número 2
-----
ID: PAC-R4AGJH6WMCPM
Nombre: B*** E*****
RUT: *****-1
Edad: 44
Correo: b*****@example.com
Fecha de registro: 2026-07-21T02:46:28.395246+00:00

Paciente número 3
-----
ID: PAC-P2D7PII3CXRI
Nombre: C*** D***
RUT: *****-2
Edad: 26
Correo: c*****@example.com
Fecha de registro: 2026-07-21T02:46:28.410875+00:00

```

Figura 2. Reporte anonimizado sin diagnóstico.

7. Evidencias del funcionamiento (continuación)

```

Evidencia - JSON y SHA-256

ARCHIVO datos/pacientes.json
=====
[
  {
    "id_paciente": "PAC-0JHTECCGA9KF",
    "nombre": "Ana Prueba",
    "rut": "12345678-5",
    "edad": 31,
    "correo": "ana.prueba@example.com",
    "diagnostico": "Control preventivo",
    "fecha_registro": "2026-07-21T02:46:28.363996+00:00"
  },
  {
    "id_paciente": "PAC-R4AGJH6WMCPM",
    "nombre": "Bruno Ejemplo",
    "rut": "11111111-1",
    "edad": 44,
    "correo": "bruno.ejemplo@example.com",
    "diagnostico": "Revisión general",
    "fecha_registro": "2026-07-21T02:46:28.395246+00:00"
  },
  {
    "id_paciente": "PAC-P2D7PII3CXRI",
    "nombre": "Carla Demo",
    "rut": "22222222-2",
    "edad": 26,
    "correo": "carla.demo@example.com",
    "diagnostico": "Evaluación de rutina",
    "fecha_registro": "2026-07-21T02:46:28.410875+00:00"
  }
]

ARCHIVO datos/pacientes.sha256
=====
2f6ad234dead6cfe5f7b3b0758f8fd2098646a060a1d9b28b6ce0a35cc156543

```

Figura 3. Archivo JSON ficticio y resumen SHA-256.

7.1 Pruebas negativas

Se probaron una edad negativa, una edad decimal, un RUT inválido, un correo inválido, un nombre vacío y un diagnóstico demasiado largo. Las entradas fueron rechazadas de forma controlada.

```

Evidencia - pruebas negativas

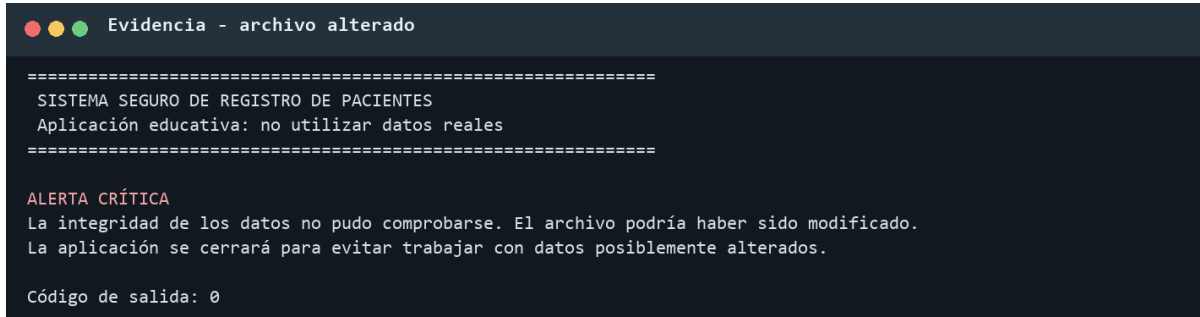
PRUEBAS NEGATIVAS
=====
[RECHAZADO] Edad negativa: La edad debe contener solo números enteros.
[RECHAZADO] Edad decimal: La edad debe ser un número entero.
[RECHAZADO] RUT inválido: El RUT ingresado no es válido.
[RECHAZADO] Correo inválido: El correo electrónico no tiene un formato válido.
[RECHAZADO] Nombre vacío: El nombre no puede estar vacío.
[RECHAZADO] Diagnóstico demasiado largo: El diagnóstico no puede superar 150 caracteres.

```

Figura 4. Resultado de las entradas inválidas.

7.2 Integridad del archivo

Después de guardar pacientes se modificó una letra del JSON de prueba sin actualizar el archivo SHA-256. Al volver a iniciar, el sistema detectó la diferencia y se cerró para no trabajar con información posiblemente alterada.



```
Evidencia - archivo alterado

=====
SISTEMA SEGURO DE REGISTRO DE PACIENTES
Aplicación educativa: no utilizar datos reales
=====

ALERTA CRÍTICA
La integridad de los datos no pudo comprobarse. El archivo podría haber sido modificado.
La aplicación se cerrará para evitar trabajar con datos posiblemente alterados.

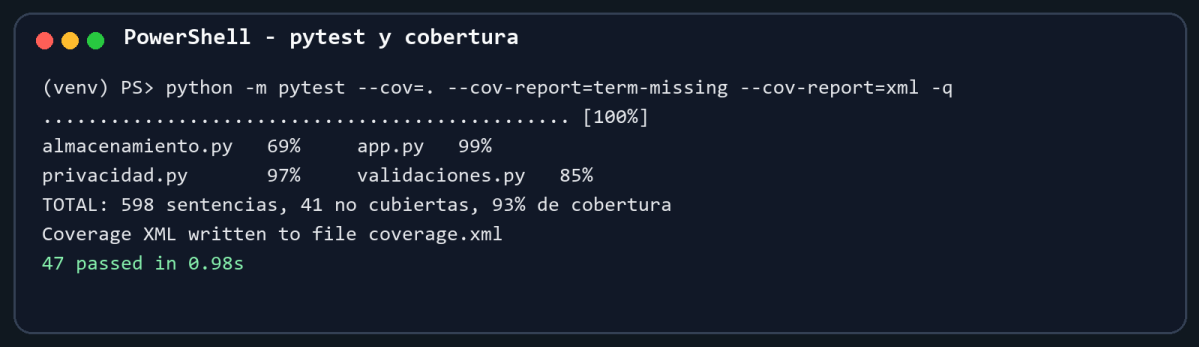
Código de salida: 0
```

Figura 5. Alerta crítica después de modificar el JSON.

Interpretación: La detención afecta la disponibilidad, pero en este caso es una decisión preventiva para proteger la integridad de los datos.

8. Pruebas automatizadas y cobertura

La suite contiene pruebas de validación, privacidad, almacenamiento y menú. Se ejecutó con Python 3.11 y pytest 8.4.2. Las 47 pruebas terminaron correctamente.



```

PowerShell - pytest y cobertura

(venv) PS> python -m pytest --cov=. --cov-report=term-missing --cov-report=xml -q
..... [100%]
almacenamiento.py 69%    app.py 99%
privacidad.py     97%    validaciones.py 85%
TOTAL: 598 sentencias, 41 no cubiertas, 93% de cobertura
Coverage XML written to file coverage.xml
47 passed in 0.98s
  
```

Figura 6. Registro de ejecución de pytest y cobertura.

Módulo	Cobertura
almacenamiento.py	69 %
app.py	99 %
privacidad.py	97 %
validaciones.py	85 %
Total local	93 %

La cobertura indica qué líneas fueron ejecutadas, pero no garantiza por sí sola la seguridad. Por eso se complementó con pruebas negativas, Bandit, SonarQube y revisión manual.

9. Resultado de Bandit

```
PowerShell - Bandit

(env) PS> python -m bandit -r . -x .\env,.\.env,.\.tests
Code scanned: 474 lines of code
Total issues: 0
Severity: Low 0 | Medium 0 | High 0
Confidence: Low 0 | Medium 0 | High 0
Resultado: 0 hallazgos
```

Figura 7. Resumen del análisis de Bandit.

Dato	Resultado
Hallazgos	0
Severidad	0 bajos, 0 medios y 0 altos
Confianza	No aplica, porque no hubo hallazgos
Líneas de código analizadas	474
Correcciones por Bandit	Ninguna
Falsos positivos	No se observaron

Bandit no reemplaza las pruebas. Un ejemplo fue la ubicación de `compare_digest`: el problema se detectó al ejecutar el sistema, se cambió a `hmac.compare_digest` y luego se agregó una prueba de integridad.

9.1 Automatización en GitHub Actions

El flujo automático instala las dependencias, ejecuta pytest con cobertura, genera el informe de Bandit y envía el análisis a SonarQube Cloud en cada actualización de la rama principal.

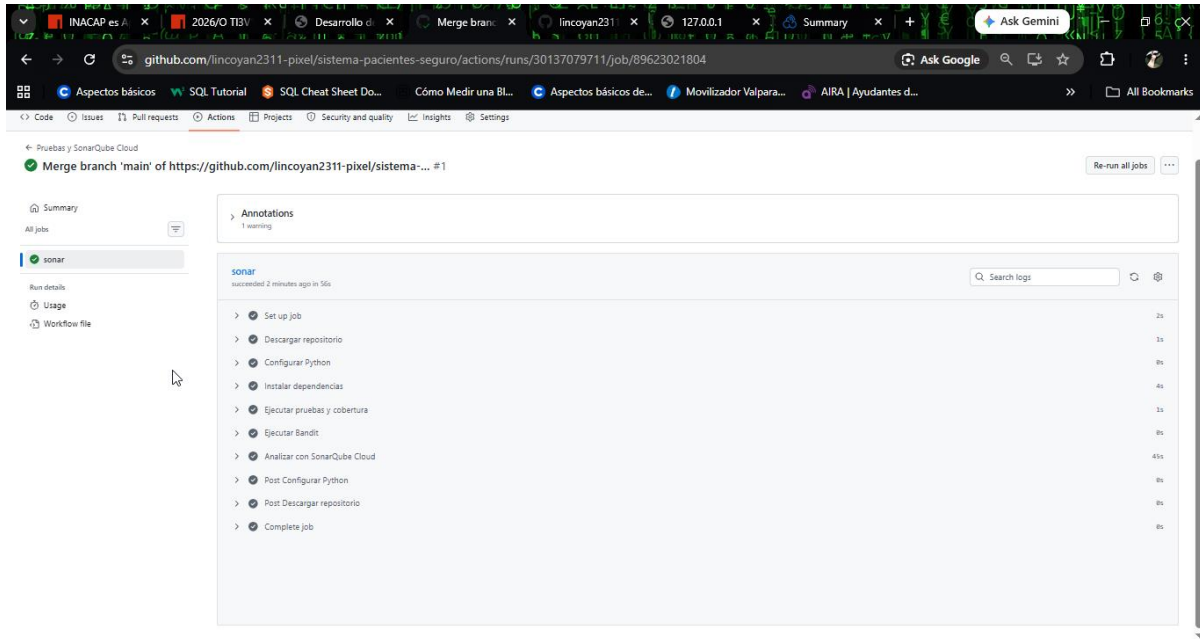


Figura 8. Flujo de GitHub Actions finalizado correctamente.

10. Resultado de SonarQube Cloud

El proyecto se analizó desde GitHub Actions utilizando un token guardado como secreto del repositorio. El token no se escribió en el código, en los archivos del proyecto ni en las capturas.

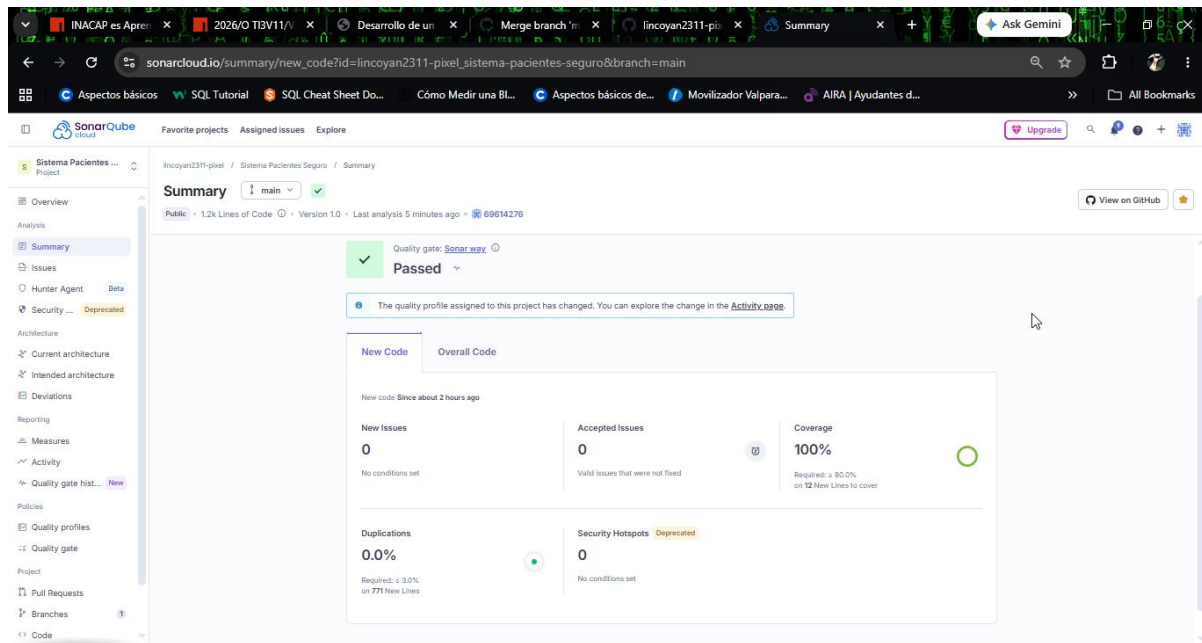


Figura 9. Quality Gate aprobado en SonarQube Cloud.

Métrica	Resultado
Quality Gate	Aprobado
Nuevos problemas	0
Security Hotspots en código nuevo	0
Cobertura de código nuevo	100 %
Duplicación de código nuevo	0,0 %
Cobertura local importada	93 % en coverage.xml

El panel corresponde a la rama main y al commit 6961427. SonarQube Cloud informó que el proyecto cumplió las condiciones del Quality Gate para el código nuevo.

11. Registro de uso responsable de GitHub Copilot

Se utilizó GitHub Copilot Free desde el navegador. La solicitud no contenía nombres, RUT, diagnósticos reales, contraseñas ni tokens.

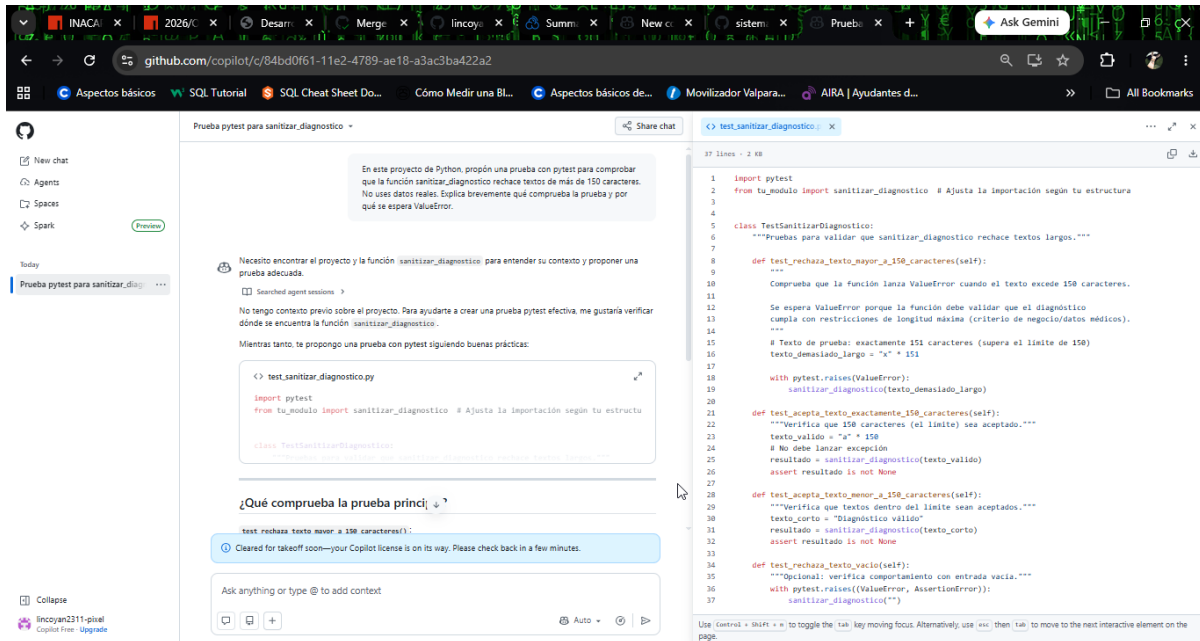


Figura 10. Prompt y propuesta de prueba generada por GitHub Copilot.

11. Registro de uso responsable de GitHub Copilot (continuación)

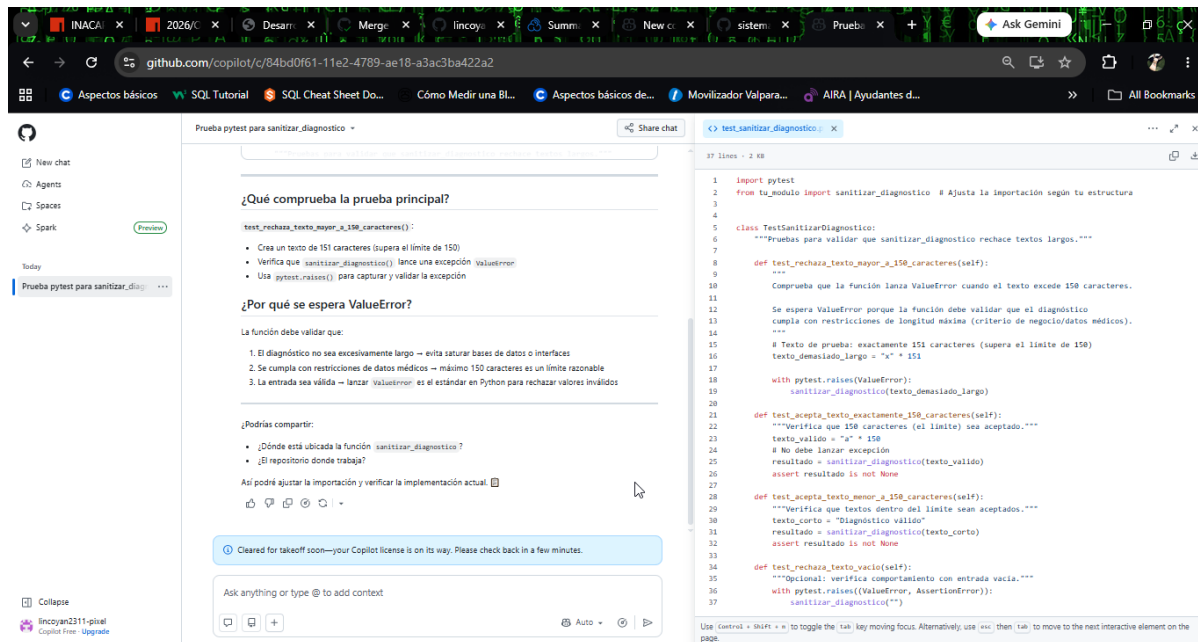


Figura 11. Explicación entregada por Copilot sobre la prueba propuesta.

Elemento	Registro
Herramienta	GitHub Copilot Free
Necesidad	Proponer una prueba para diagnósticos de más de 150 caracteres.
Prompt	Solicitar una prueba pytest, sin datos reales, y explicar por qué se espera ValueError.
Propuesta principal	Crear un texto de 151 caracteres y comprobar <code>pytest.raises(ValueError)</code> .
Problema detectado	La importación «from tu_modulo» era solamente un marcador y no era válida para este proyecto.
Comparación	El caso principal ya estaba cubierto por <code>test_diagnostico_demasiado_largo_genera_error</code> .
Decisión	No se copió el archivo completo para evitar pruebas duplicadas; se mantuvo la prueba existente.
Verificación	La prueba equivalente se ejecutó dentro de la suite y el resultado fue 47 passed.

Conclusión de la revisión: Copilot fue utilizado como apoyo. La propuesta se entendió, se comparó con el proyecto y no se aceptó automáticamente.

12. Selección segura de dependencias

Dependencia	Uso	Criterio
Biblioteca estándar	Aplicación	No se agregaron paquetes externos para ejecutar el programa.
pytest 8.4.2	Pruebas	Paquete conocido, nombre exacto y versión fijada.
pytest-cov 7.0.0	Cobertura	Complemento específico de pytest.
Bandit 1.9.4	Análisis	Herramienta orientada a seguridad en código Python.

requirements-lock.txt registra las dependencias transitivas. Antes de instalar se revisa el nombre exacto, la fuente, el mantenimiento, la licencia y si el paquete realmente es necesario. Esto reduce el riesgo de typosquatting.

13. Reflexión sobre SAMM y SDL

SAMM nos ayudó a ordenar el trabajo en gobierno, diseño, implementación, verificación y operaciones. SDL permitió relacionar esos controles con las etapas del desarrollo: primero se definieron requisitos y riesgos, después se programó y finalmente se verificó con varias herramientas.

La principal mejora respecto del primer análisis fue automatizar pytest, cobertura, Bandit y SonarQube mediante GitHub Actions. Aun así, el proyecto sigue siendo un prototipo académico y no un sistema clínico.

Controles necesarios para producción

- Cifrado de datos y gestión segura de claves.
- Autenticación, segundo factor, roles y permisos.
- Base de datos transaccional, auditoría y control de concurrencia.
- Respallos, restauración probada y monitoreo.
- Revisión legal y cumplimiento de normas de protección de datos.
- Plan de respuesta a incidentes y revisión continua de dependencias.

14. Revisión ética

Comprobación	Respuesta
¿Se usaron datos reales?	No
¿Se enviaron secretos o tokens a una IA?	No
¿Se aceptó código sin revisarlo?	No
¿Se ejecutaron pruebas, Bandit y SonarQube?	Sí
¿Se revisaron manualmente los resultados?	Sí

15. Respuestas a las preguntas de análisis

Las respuestas siguientes resumen los conceptos utilizados durante el desarrollo y la verificación del proyecto.

Pregunta	Respuesta
1. ¿Cuál es la diferencia entre validar y sanitizar?	Validar comprueba si un dato cumple las reglas y puede rechazarlo. Sanitizar limpia o transforma la entrada para dejar contenido permitido.
2. ¿Por qué no debe almacenarse directamente el dato ingresado?	Porque la entrada del usuario no es confiable: puede estar vacía, fuera de rango, mal formada o contener caracteres no permitidos.
3. ¿Por qué secrets es más apropiado que random para identificadores?	secrets utiliza una fuente de aleatoriedad adecuada para seguridad y produce valores menos predecibles. random está pensado para simulaciones.
4. ¿La seudonimización elimina todos los riesgos de privacidad?	No. Si existe información adicional que permita relacionar el seudónimo, todavía se podría identificar a la persona.
5. ¿Ocultar parte del RUT equivale a anonimización irreversible?	No. Es un enmascaramiento visual; el RUT completo continúa almacenado y podría relacionarse con otros datos.

15. Respuestas a las preguntas de análisis (continuación)

Pregunta	Respuesta
6. ¿Qué principio CIA se afecta si un archivo es modificado?	La integridad, porque ya no se puede confiar en que su contenido sea correcto.
7. ¿Qué principio CIA se afecta si el sistema se detiene?	La disponibilidad, porque los usuarios no pueden utilizar el servicio.
8. ¿Qué principio CIA se afecta si se muestra el diagnóstico?	La confidencialidad, porque se expone un dato sensible a personas que podrían no estar autorizadas.
9. ¿Por qué no basta con instalar una librería desde PyPI?	También se debe revisar el nombre, mantenedor, documentación, licencia, versiones, dependencias y vulnerabilidades conocidas.
10. ¿Qué es un ataque de typosquatting?	Es publicar un paquete con un nombre parecido al original para aprovechar errores de escritura e instalar código malicioso.

15. Respuestas a las preguntas de análisis (continuación)

Pregunta	Respuesta
11. ¿Qué diferencia existe entre Bandit y SonarQube?	Bandit busca patrones de seguridad en Python. SonarQube analiza calidad general, bugs, vulnerabilidades, hotspots, duplicación y cobertura.
12. ¿Qué es un falso positivo?	Es un hallazgo que parece un problema, pero después de revisar el contexto se determina que no representa un riesgo real.
13. ¿Por qué un Security Hotspot requiere revisión humana?	Porque señala una zona sensible, pero la herramienta no conoce todo el contexto. Una persona debe decidir si el control es suficiente.
14. ¿Una cobertura del 100 % garantiza seguridad?	No. Solo indica que ciertas líneas fueron ejecutadas; las pruebas pueden seguir siendo incompletas.
15. ¿Por qué no se debe confiar automáticamente en GitHub Copilot?	Porque puede proponer código incorrecto, desactualizado o inseguro. La sugerencia debe comprenderse y probarse.

15. Respuestas a las preguntas de análisis (continuación)

Pregunta	Respuesta
16. ¿Quién es responsable del código generado por IA?	La persona que decide incorporarlo, ejecutarlo y entregarlo. La herramienta no reemplaza la responsabilidad del desarrollador.
17. ¿Qué información nunca debería enviarse a una IA pública?	Datos personales o médicos reales, contraseñas, tokens, claves privadas y código confidencial sin autorización.
18. ¿Por qué el token de SonarQube debe protegerse?	Porque permite autenticarse. Si se filtra, otra persona podría utilizar los permisos asociados al token.
19. ¿Qué controles faltarían para utilizar el sistema en producción?	Cifrado, autenticación, roles, auditoría, respaldos, base de datos, monitoreo, cumplimiento legal y respuesta a incidentes.
20. ¿Cómo ayuda SAMM a mejorar la seguridad de una organización?	Permite evaluar prácticas, definir un nivel actual, priorizar mejoras y avanzar gradualmente con actividades medibles.

16. Conclusiones

La actividad permitió comprobar que la seguridad puede incorporarse desde el comienzo incluso en un programa sencillo. Las validaciones evitaron datos incoherentes, la sanitización limitó caracteres, el reporte redujo la exposición y el hash permitió detectar una modificación manual.

Las pruebas automatizadas, Bandit y SonarQube Cloud entregaron evidencias diferentes. Ninguna herramienta fue tomada como garantía absoluta. GitHub Copilot también se utilizó con revisión humana, comparando su propuesta con las pruebas existentes antes de decidir.

17. Limitaciones

- El archivo JSON se almacena en texto claro y no existe cifrado en reposo.
- No hay autenticación, roles ni autorización.
- El hash no utiliza una clave secreta y se guarda junto al archivo.
- La anonimización es básica y podría no ser irreversible.
- No existe base de datos, auditoría, respaldo automático ni control de concurrencia.
- El proyecto es un prototipo académico y no debe utilizarse con información clínica real.

18. Propuestas de mejora

- Usar HMAC con una clave almacenada fuera del código.
- Cifrar los datos y respaldos con una gestión segura de claves.
- Agregar autenticación, segundo factor, roles y permisos.
- Migrar a una base de datos con consultas parametrizadas.
- Incorporar auditoría sin registrar diagnósticos ni identificadores completos.
- Agregar revisión continua de vulnerabilidades en las dependencias.

19. Lista de comprobación final

Comprobación	Estado
Programa y menú	Cumplido
Validación, sanitización y anonimización	Cumplido
JSON, SHA-256 y detección de cambios	Cumplido
Pruebas automatizadas	47 aprobadas
Cobertura XML	93 % local
Bandit JSON	0 hallazgos
SonarQube Cloud	Quality Gate aprobado
Dependencias fijadas	Cumplido
Uso de GitHub Copilot documentado	Cumplido
Sin datos reales, tokens ni entorno virtual en la entrega	Cumplido

Anexo. Comandos principales

```
python -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install -r requirements-dev.txt
python app.py
python -m pytest -v
python -m pytest --cov=. --cov-report=term-missing --cov-report=xml
python -m bandit -r . -x .\venv,.\.venv,.\tests -f json -o reporte-bandit.json
git status
git add .
git commit -m "Actualizar evidencias finales"
git push
```

Fuente de la actividad

Guía práctica «Desarrollo de un sistema seguro de registro de pacientes en Python», material entregado para la asignatura Introducción a la Programación Segura.

Repositorio del proyecto: <https://github.com/lincoyan2311-pixel/sistema-pacientes-seguro>

Panel de análisis: https://sonarcloud.io/summary/new_code?id=lincoyan2311-pixel_sistema-pacientes-seguro&branch=main